

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14
Questions

COURSE NAME: Compiler Design

Annexure A

ERROR ANALYSIS TASK

Q1. The following C program is intended to perform arithmetic operations, function calls, array processing, conditional statements, and loop execution. However, the code contains multiple errors introduced at different phases of compilation, including lexical, syntax, semantic, logical, and runtime errors.

Given Program

```
#include <stdio.h>
```

```
int square(int n)
```

```
{
```

```
    return n * n;
```

```
}
```

```
int main()
```

```
{
```

```
    int 2num = 10;
```

```
    int arr[5] = {1, 2, 3, 4, 5}
```

```
    int result;
```

```
    result = square();
```

```

if(result = 100)
{
    printf("Result is 100\n")
}

total = result + 10;

printf("%d\n", arr[5]);

int *p;
*p = 50;

int x = 20, y = 0;
int z = x / y;

int i = 1;
while(i <= 5)
{
    printf("%d\n", i);
}

return 0;
}

```

Tasks

(a) Error Identification (Analysis Level)

Identify all the errors present in the program. For each error:

Specify the line number.

Clearly describe the nature of the error.

Explain why the statement is incorrect.

(b) Classification of Errors (Understanding + Analysis)

Classify each identified error into the appropriate compiler phase:

Lexical Errors

Syntax Errors

Semantic Errors

Logical Errors

Runtime Errors

Present the classification in a tabular format.

(c) Compilation Behavior (Evaluation Level)

Analyze the compilation process for the given program.

Clearly explain:

Which errors are detected during lexical analysis.

Which errors are detected during syntax analysis.

Which errors are detected during semantic analysis.

Which errors appear only during execution.

How earlier errors may prevent the compiler from detecting later errors.

(d) Code Correction (Application Level)

Rewrite the corrected version of the program such that:

All errors are removed.

Proper syntax and semantics are maintained.

Array accesses remain within valid bounds.

Pointer operations are safe.

The loop terminates correctly.

The program executes successfully.

(e) Impact Analysis (Evaluation Level)

Discuss the impact of at least three errors on program execution.

Explain how the following affect system behavior:

Dereferencing an uninitialized pointer.

Division by zero.

Array index out-of-bounds access.

Infinite loop execution.

Incorrect use of assignment operator inside a condition.

(f) Advanced Compiler Perspective (Create Level)

Suggest how a compiler can be improved to:

Generate more meaningful error messages.

Detect uninitialized variables.

Warn about possible division by zero.

Detect array boundary violations.

Identify potential infinite loops. (20 Marks)

Q2. The following C program contains multiple syntax errors introduced intentionally. These errors occur due to violations of the grammatical rules of the C language, such as missing punctuation symbols, unmatched braces, incorrect statement structures, and malformed control constructs. These errors are typically detected during the syntax analysis (parsing) phase of a compiler. Carefully examine the program and answer the questions given below.

(10 Marks)

Given Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a = 10
```

```
    int b = 20;
```

```
    if(a < b
```

```
    {
```

```
        printf("A is smaller\n");
```

```
    }
```

```
    for(int i = 0; i < 5; i++
```

```
    {
```

```
        printf("%d\n", i);
```

```
    }
```

```
    while(a < 20
```

```
    {
```

```
        a++;
```

```
}
```

```
printf("Sum = %d\n", a + b;
```

```
return 0;
```

Tasks

(a) Error Identification

Identify all syntax errors present in the program.

List the line number and the incorrect statement.

Explain why each statement violates the syntax rules of C.

(b) Error Classification

For each identified error:

Specify the violated grammar rule.

Classify the syntax error into one of the following categories:

Missing delimiter errors

Unmatched parentheses/braces

Incorrect control structure syntax

Missing statement terminators

Incomplete function definitions

(c) Syntax Correction

Rewrite each incorrect statement into its correct form.

Provide the corrected version of the entire program.

Example:

```
int a = 10
```

Corrected as:

```
int a = 10;
```

(d) Compiler Behavior Analysis

Explain how the parser detects syntax errors.

Discuss the role of Context-Free Grammars (CFGs) in syntax analysis.

Explain how a parser reports:

Missing semicolons

Missing braces

Unmatched parentheses

Discuss whether:

The parser stops immediately after detecting an error.

Error recovery techniques allow further parsing.

(e) Advanced Thinking

Design a simple grammar rule for an if statement.

Example:

$\text{if_statement} \rightarrow \text{if (condition) statement}$

Explain how a parser would detect a missing closing parenthesis in an if statement.

Suggest two improvements in parser design to provide more user-friendly syntax error messages for beginners. (15 Marks)

Q3. The following C program is designed to perform array processing, arithmetic calculations, function calls, conditional execution, and loop operations. However, the program contains multiple errors distributed across different phases of compilation and execution, including:

Lexical Errors

Syntax Errors

Semantic Errors

Intermediate Code Generation Issues

Code Optimization Issues

Runtime Errors

Carefully analyze the program and answer the questions below. (10 Marks)

Given Program

```
#include <stdio.h>
```

```
int square(int n)
```

```
{
```

```
    return n * n;
```

```
}
```

```
int main()
```

```
{
    int #count = 5;

    float rate = 2.5

    int result;

    result = square();

    if(result > 20)
    {
        printf("Large Value\n");
    }
    else
    {
        printf("Small Value\n");
    }

    int x = 100;
    int y = 0;
    int z = x / y;

    int marks[4] = {10, 20, 30, 40};
    printf("%d\n", marks[4]);

    int *ptr;
    *ptr = 25;

    int total = x + 0 + y;

    int i = 1;
    while(i <= 5)
    {
        printf("%d\n", i);
    }
}
```

```
    return 0;  
}
```

Tasks

(a) Identify Errors (Analysis Level)

Identify all errors present in the program.

Mention the line number and describe each error.

Explain why the statement is incorrect.

(b) Classify by Compiler Phase

Classify each identified error into the appropriate compiler phase:

Lexical Errors

Invalid identifiers

Illegal symbols

Syntax Errors

Missing semicolons

Missing braces

Incorrect statement structure

Semantic Errors

Function argument mismatch

Type-related issues

Undeclared identifiers

Intermediate Code Generation Issues

Incorrect expression evaluation

Invalid operand usage

Optimization Issues

Redundant arithmetic operations

Unnecessary computations

Runtime Errors

Division by zero

Invalid memory access

Array bounds violation

(c) Compilation vs Execution Analysis (Evaluation Level)

Analyze the behavior of the compiler and runtime system.

Explain:

Which errors are detected during lexical analysis?

Which errors are detected during syntax analysis?

Which errors are detected during semantic analysis?

Which errors appear only during execution?

How lexical and syntax errors can prevent later compiler phases from executing.

(d) Program Correction (Application Level)

Rewrite the complete corrected version of the program such that:

All lexical errors are removed.

All syntax errors are corrected.

Function calls are semantically valid.

Array accesses remain within bounds.

Pointer usage is safe.

Division-by-zero situations are avoided.

The loop terminates correctly.

(e) Deep Analysis (Evaluation Level)

Discuss the impact of the following errors on program execution:

Dereferencing an uninitialized pointer.

Accessing an array element outside its valid range.

Infinite loop execution.

Division by zero.

Missing function arguments.

Also explain how these errors may affect:

Memory safety

Program correctness

System stability

(f) Advanced Compiler Perspective (Create Level)

Suggest at least two enhancements in compiler design to improve program reliability.

Your suggestions should address:

Better compile-time detection of runtime errors.

Improved diagnostic and debugging messages.

Detection of possible infinite loops using static analysis.

Detection of array bound violations.

Detection of uninitialized pointer usage.

Optimization techniques that remove redundant expressions such as:

`total = x + 0 + y;`

Explain how the optimization phase can simplify the above expression and generate more efficient target code. (15 Marks)

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand